UNIVERSIDADE CIDADE DE SÃO PAULO - UNICID

CIÊNCIA DA COMPUTAÇÃO

GUSTAVO OLIVEIRA CASTRO - RGM 34032797
KAIO DO NASCIMENTO MENDES - RGM 33604410

PIPELINE DE DADOS EM HASKELL: MAP, FILTER E FOLD SOBRE REGISTROS

São Paulo
2026

GUSTAVO OLIVEIRA CASTRO - RGM 34032797

KAIO DO NASCIMENTO MENDES - RGM 33604410

PIPELINE DE DADOS EM HASKELL: MAP, FILTER E FOLD SOBRE REGISTROS

Relatório apresentado à disciplina Paradigmas de Linguagens de Programação, do curso de Ciência da Computação da Universidade Cidade de São Paulo - UNICID, como parte da avaliação sobre paradigmas de programação.

Professor: Cid Rodrigues de Andrade.

São Paulo

2026

SUMÁRIO

1 INTRODUÇÃO 4

2 DESCRIÇÃO DA SOLUÇÃO 5

3 COMO EXECUTAR 6

4 ENTRADA E SAÍDA DE EXEMPLO 7

5 ANÁLISE COMPARATIVA: HASKELL E PYTHON 9

6 LOG DE USO DE INTELIGÊNCIA ARTIFICIAL 11

7 CONCLUSÃO 12

REFERÊNCIAS 13

1 INTRODUÇÃO

Este relatório apresenta uma solução em Haskell para processar dados acadêmicos de uma turma. O programa calcula nota final e frequência, classifica cada aluno como aprovado ou reprovado, calcula estatísticas e produz um relatório. O objetivo é demonstrar um pipeline declarativo com map, filter e foldl sobre registros.

A solução considera carga horária de 60 horas, com cada falta equivalente a uma hora. A aprovação exige nota final igual ou superior a 6,0 e frequência igual ou superior a 75%.

2 DESCRIÇÃO DA SOLUÇÃO

2.1 Modelagem dos dados

Aluno contém nome, notas A1 e A2 e faltas. Status restringe a classificação aos valores Aprovado e Reprovado. Resultado contém nome, nota final, frequência e situação. O tipo próprio Status evita textos inválidos e melhora a segurança do programa.

```
data Aluno = Aluno { nome :: String, notaA1 :: Double, notaA2 :: Double, faltas ::
Int } deriving (Show)

data Status = Aprovado | Reprovado deriving (Show, Eq)

data Resultado = Resultado { nomeAluno :: String, notaFinal :: Double, frequencia ::
Double, situacao :: Status } deriving (Show)
```

2.2 Processamento funcional

avaliarAluno soma as notas, calcula a frequência e aplica as regras de aprovação. map transforma cada Aluno em Resultado; filter separa aprovados e reprovados; foldl soma notas e frequências para as médias. As funções de média retornam zero para listas vazias, evitando divisão por zero.

```
resultados = map avaliarAluno alunos

aprovados = filter ((== Aprovado) . situacao) resultados

reprovados = filter ((== Reprovado) . situacao) resultados
```

3 COMO EXECUTAR

O código foi desenvolvido para execução em um ambiente on-line gratuito, tendo como foco o TIO - Try It Online (tio.run). Para executar, basta selecionar a linguagem Haskell, colar o código completo no editor e acionar o comando de execução. Não é necessário instalar compiladores, bibliotecas ou outras ferramentas no computador.

Não é necessário fornecer dados pelo teclado, pois a entrada está declarada na lista alunos. Para testar outros casos, basta substituir ou editar os registros dessa lista, preservando a ordem dos campos: nome, nota A1, nota A2 e número de faltas.

O código completo, o permalink do TIO e os arquivos da entrega estão disponíveis no repositório do grupo no GitHub:

Acessar o repositório paradigma-funcional-grupo08 no GitHub

4 ENTRADA E SAÍDA DE EXEMPLO

4.1 Entrada utilizada

```
alunos =
 [ Aluno "Mariana Costa" 3.0 3.0 15
 , Aluno "Rafael Martins" 4.5 4.0 3
 , Aluno "Beatriz Ramos" 2.5 3.0 10
 , Aluno "Caio Ferreira" 4.0 3.5 16
 , Aluno "Larissa Gomes" 2.0 3.0 20
 , Aluno "Pedro Almeida" 5.0 5.0 0
 , Aluno "Isabela Nunes" 3.5 2.5 12
 , Aluno "Thiago Ribeiro" 1.5 2.0 25
 , Aluno "Sofia Carvalho" 4.0 3.0 15
 , Aluno "Matheus Barbosa" 2.8 3.1 5 ]
```

4.2 Saída documentada

```
RELATÓRIO
DADOS DA TURMA
Carga horária considerada  : 60 horas
Total de alunos analisados : 10
Quantidade de aprovados    : 5
Quantidade de reprovados   : 5
Média de notas da turma    : 6.5
Média de frequência geral  : 79.8%

ALUNOS APROVADOS
- Mariana Costa [Nota Final: 6.0 | Frequência: 75.0%]
- Rafael Martins [Nota Final: 8.5 | Frequência: 95.0%]
- Pedro Almeida [Nota Final: 10.0 | Frequência: 100.0%]
- Isabela Nunes [Nota Final: 6.0 | Frequência: 80.0%]
- Sofia Carvalho [Nota Final: 7.0 | Frequência: 75.0%]

ALUNOS REPROVADOS
```

- Beatriz Ramos [Nota Final: 5.5 | Frequência: 83.3%]
- Caio Ferreira [Nota Final: 7.5 | Frequência: 73.3%]
- Larissa Gomes [Nota Final: 5.0 | Frequência: 66.7%]
- Thiago Ribeiro [Nota Final: 3.5 | Frequência: 58.3%]
- Matheus Barbosa [Nota Final: 5.9 | Frequência: 91.7%]

FIM DO RELATÓRIO

5 ANÁLISE COMPARATIVA: HASKELL E PYTHON

A comparação utiliza Python em estilo imperativo, com laços, listas modificáveis e acumuladores. Haskell descreve transformações sobre listas; Python detalha as instruções para produzir cada resultado.

5.1 Sintaxe

Em Haskell, map aplica uma função a toda a lista em uma expressão. Em Python, cria-se uma lista vazia, percorrem-se os alunos com for e adicionam-se resultados com append. filter corresponde a for, if e append. foldl corresponde a um laço que atualiza uma variável acumuladora.

```
-- Haskell
resultados = map avaliarAluno alunos


# Python imperativo
resultados = []
for aluno in alunos:
    resultados.append(avaliar_aluno(aluno))
```

5.2 Semântica

Haskell possui tipagem estática e inferência de tipos. Assinaturas são verificadas antes da execução e Status admite apenas Aprovado ou Reprovado. Python possui tipagem dinâmica e suas anotações não oferecem, por padrão, as mesmas garantias. Em Haskell, nomes representam valores imutáveis; em Python, atribuições vinculam nomes a objetos e listas podem ser alteradas. A unificação participa da inferência de tipos em Haskell, enquanto a atribuição atualiza vinculações em Python.

Haskell possui semântica não estrita, normalmente implementada por avaliação preguiçosa. Python normalmente avalia argumentos e executa laços de forma estrita. Haskell explicita efeitos com IO, mantendo avaliarAluno e as médias como funções puras; Python não separa efeitos no sistema de tipos.

5.3 Gerenciamento de memória

As duas linguagens oferecem gerenciamento automático de memória. Haskell favorece valores imutáveis e pode adiar cálculos; Python permite objetos mutáveis. Em listas grandes, foldl pode acumular expressões pendentes e foldl' pode ser preferível. Em Python, append modifica uma lista existente. O contraste principal neste problema é imutabilidade contra mutabilidade, e não liberação manual de memória.

5.4 Trade-offs

Haskell oferece concisão, segurança de tipos, funções puras e menor risco de alterações inesperadas. Como desvantagens, exige familiaridade com funções de ordem superior, composição, IO e avaliação preguiçosa. Python é mais familiar para quem conhece for, if e acumuladores e apresenta um fluxo passo a passo; porém, exige mais código de controle, permite efeitos colaterais e oferece menos garantias estáticas sem ferramentas adicionais.

Aspecto	Haskell	Python imperativo
Fluxo	map, filter e foldl	for, if, append e acumuladores
Estado	Imutável	Mutável
Tipos	Estáticos, com inferência	Dinâmicos
Avaliação	Preguiçosa	Geralmente estrita
Memória	Automática	Automática

6 LOG DE USO DE INTELIGÊNCIA ARTIFICIAL

Durante o desenvolvimento do trabalho, foi solicitado à ferramenta de inteligência artificial que explicasse o funcionamento de map, filter e foldl em Haskell, revisasse o código produzido pelo grupo e propusesse casos de teste para as regras de aprovação e reprovação. Da revisão, foram aproveitadas as sugestões de criar as funções arredondarUmaCasa, para apresentar valores com uma casa decimal, e formatarLinhaAluno, para centralizar a montagem das linhas do relatório e evitar repetição de código. Também foram aproveitados casos de teste que representam aprovação exatamente nos limites, reprovação somente por nota, reprovação somente por frequência e reprovação pelos dois critérios. As explicações e sugestões não foram utilizadas de forma automática: o grupo revisou o conteúdo, reescreveu os comentários e as justificativas com suas próprias palavras e verificou o funcionamento das funções e dos casos de teste na execução do programa. Dessa forma, a inteligência artificial foi utilizada como apoio para compreender conceitos, revisar a solução e ampliar os testes, enquanto as decisões finais e a compreensão do código permaneceram sob responsabilidade do grupo.

7 CONCLUSÃO

A solução demonstrou como registros e funções de ordem superior podem organizar um pipeline acadêmico em Haskell. map foi utilizado para transformar alunos em resultados, filter para separar aprovados e reprovados e foldl para agregar notas e frequências. O uso de tipos próprios tornou o domínio mais explícito, especialmente ao limitar o status a duas alternativas válidas.

A comparação com Python mostrou que ambas as linguagens resolvem o mesmo problema, mas incentivam formas distintas de raciocínio. O estilo imperativo descreve instruções, atualizações e acumuladores; o estilo funcional descreve transformações e combinações de funções. Para este caso, Haskell forneceu uma solução concisa e segura, embora apresente uma curva de aprendizagem maior para quem está habituado à programação imperativa.

REFERÊNCIAS

LIMA, Maria Adriana Vidigal de. Programação funcional em Haskell: funções genéricas sobre listas. Uberlândia: Faculdade de Computação da Universidade Federal de Uberlândia, 2009. Disponível em: https://www.facom.ufu.br/~madriana/PF/AulaCap9.pdf. Acesso em: 2 out. 2026.

CAMARÃO, Carlos; FIGUEIREDO, Lucília. Funções de ordem superior. In: Programação funcional. Belo Horizonte: Departamento de Ciência da Computação da Universidade Federal de Minas Gerais, [s. d.]. Disponível em: https://homepages.dcc.ufmg.br/~camarao/haskell/livro005.html. Acesso em: 7 out. 2026.

UNIVERSIDADE FEDERAL DO ABC. Paradigmas de programação. In: Introdução à programação funcional em Haskell. Santo André: Grupo de Estudos em Haskell da UFABC, [s. d.]. Disponível em: https://haskell.pesquisa.ufabc.edu.br/haskell/01.paradigmas/. Acesso em: 12 out. 2026.